DATA 200: Python Learning Log

A Semester Record of Python Practice and Growth

Jason E. Czak

Abstract

The Python Learning Log asks you to document regular Python practice: code development, debugging, testing, and reflection. You complete at least four substantive entries per week. Each entry pairs a paragraph about what you tried and learned with its own uniquely named Python file, the complete code in a LaTeX report, and a screenshot showing that the program ran on your own computer.

Contents

1 Assignment Purpose **2**

2 What Each Entry Includes **2**

3 Getting Started **3**

4 Repository Organization **4**

5 Checking Your Work **4**

6 Assessment Evidence **4**

7 Submission **5**

8 Academic Integrity **5**

1 Assignment Purpose

The goal of this assignment is to become more proficient and independent in Python. Over the semester you practice regularly, document both successful and unsuccessful attempts, and keep examples of the programs you create. The focus is not only on making code run, but also on understanding why it works, communicating its purpose through comments, and improving the organization and reliability of your programs. The finished log shows how your problem-solving process and programming vocabulary develop over time.

You keep this log for the whole semester. It supports several later projects and assignments: some of its entries are turned in under one of them and some under another, and the instructions for each say which entries belong with it.

2 What Each Entry Includes

The log must contain at least four substantive entries per week. Each entry includes:

- the date and the time spent;
- the learning focus and the resources used;
- a paragraph-length progress narrative: what you attempted, what you learned, where you had difficulty, what changes you made, and how you checked the result;
- one uniquely named Python source file;
- an explanation of the program's purpose, inputs, and outputs;
- the complete code in the LaTeX report;
- a screenshot showing that the program ran successfully on your own computer; and
- reflection on debugging and the skills you gained.

Be specific. Identify an error message, an incorrect output, or a design decision rather than writing only "I practiced Python." Comments in your Python files should become more detailed and technically precise as your proficiency increases.

2.1 What can count as an entry

You may document relevant homework, guided class examples, tutorial work, documentation study, debugging, and independent practice when those activities genuinely contributed to your Python learning. An entry documents active learning rather than merely stating that an assignment was completed. Whatever the source, you write your own narrative, submit your own code, and describe your own debugging and testing.

2.2 Screenshots

Every entry needs a screenshot of its program running successfully on your own computer. The screenshot is your evidence that the code you submitted actually runs, so it must come from your own run of the submitted file, not from a classmate, a tutorial, or a website. Make sure the output is readable, give each screenshot a caption, and check that it shows no passwords, tokens, or other private information.

2.3 Using AI tools

A later part of the course covers responsible use of generative AI. Until then, write your entries without AI tools. For entries written after that discussion, use AI only as the course policy allows, and document every use in the AI-Use Note section of your log: the tool, the task, and how you checked the result. Do not claim authorship or understanding of code that you cannot explain.

3 Getting Started

3.1 Software

You need the course Python environment and a working LaTeX installation from the course setup. If you cannot install software on your computer, contact the instructor for the course's approved alternative.

With conda (recommended), on Windows (Anaconda Prompt) or Linux:

```
conda env create -f environment.yml
conda activate data200
```

If you already have the `data200` environment from the course setup, you do not need to change it; this log uses the same packages.

Or with pip:

```
python -m venv .venv
.venv\Scripts\activate             (Windows)
source .venv/bin/activate          (Linux)
python -m pip install -r requirements.txt
```

3.2 Your log

Start from `report/template.tex`. It contains the sections your log needs and a Week 1 subsection with four entries to fill in. Copy the Week 1 subsection for each additional week, and update the dates, file names, listing labels, screenshot names, and captions; every `\label` must be unique. Until a screenshot file exists, the template shows a "YOUR screenshot here" box in its place, so the log compiles from the first day. Compile it from its own folder with two passes, which resolve the table of contents and page numbers:

```
pdflatex template.tex
pdflatex template.tex
```

Use file names that identify both the log entry and the purpose of the program, for example:

```
python_progress_log.tex
python_progress_log.pdf
week01_entry01_greeting.py
week01_entry01_output.png
week01_entry02_temperature_converter.py
week01_entry02_output.png
```

3.3 The fictional sample

The file `report/fictional-sample-entries.pdf` shows four completed Week 1 entries. They are fictional and model the expected level of detail: a temperature converter (variables, input, and arithmetic), a grade classifier (conditional statements), an average calculator (loops and accumulation), and a data summary (lists and introductory descriptive statistics). The examples deliberately progress from basic syntax toward a small data-analysis task. Their programs are in `src/sample_entries`.

The sample marks each screenshot with a "YOUR screenshot here" placeholder; your log shows real screenshots instead. Use the sample as a model of specificity, not as text to reuse: do not copy its narratives or present its code as your own work.

4 Repository Organization

```
(repository folder)/
|-- README.md, LICENSE, LICENSE-CONTENT.md, CITATION.cff, CHANGELOG.md
|-- environment.yml, requirements.txt, pytest.ini, .gitignore
|-- handout/          this document
|-- report/
|   |-- template.tex                  starting point for your log
|   |-- fictional-sample-entries.tex  four fictional completed entries
|   `-- screenshot-placeholder.png    the sample's placeholder image
|-- src/
|   |-- sample_entries/   one program for each fictional entry
|   `-- make_screenshot_placeholder.py
|-- notebooks/        01-worked-example: testing the sample programs
|-- figures/
|   `-- reference/    expected console output of the sample programs
`-- tests/
```

5 Checking Your Work

Test every program with values you can check by hand, including exact boundary values, before you write its run result. The notebook `notebooks/01-worked-example.ipynb` shows this for the four sample programs; open it with `jupyter lab`. To see the sample programs run, from the repository root:

```
python src/sample_entries/week01_entry01_temperature_converter.py
python src/sample_entries/week01_entry02_grade_classifier.py
python src/sample_entries/week01_entry03_average_calculator.py
python src/sample_entries/week01_entry04_data_summary.py
```

The files in `figures/reference` hold their expected output for the inputs 0, 90, and 10, 20, 30, 40, 50. Running `pytest` checks the sample programs, the notebook, and that every document compiles.

6 Assessment Evidence

A complete log makes the following evidence visible:

- **Completion and consistency:** at least four substantive entries for every required week; dates, time spent, learning focus, resources, and file names supplied; a clear and consistent naming system.
- **Progress narratives and reflection:** a developed paragraph for each entry rather than a brief activity list; attempts, difficulties, revisions, and learning explained; a final reflection that identifies growth, remaining challenges, and next steps.
- **Python programs:** its own uniquely named `.py` file for every entry; each program runs and addresses the stated learning goal; code that becomes more organized and appropriately commented over time.
- **Testing and evidence:** an explanation of how each program was tested; screenshots from your own computer showing successful execution and readable output; boundary cases, simple hand-checkable cases, or other suitable tests.
- **Skills gained:** specific Python concepts identified; claims of improvement supported with references to log entries or code.
- **Technical presentation:** a readable, organized PDF compiled from the submitted LaTeX source; code included verbatim and matching the submitted Python files; all required source, image, and data files included.

7 Submission

Before submitting, confirm that you included:

☐ the compiled PDF report and its LaTeX `.tex` source;

☐ at least four complete log entries for each required week;

☐ a unique `.py` file for every log entry, with the complete code in the report;

☐ a description of what each program does and how it was tested;

☐ a screenshot showing each program running successfully on your own computer;

☐ a discussion of skills gained, an overall reflection, and a next step;

☐ any data files required to run the programs;

☐ file names in the report that exactly match the submitted files; and

☐ no private usernames, passwords, tokens, or unrelated personal data.

You may also keep this work in your own Git repository and publish it on GitHub; that choice is yours.

8 Academic Integrity

The fictional sample is supplied only to model the expected level of detail. Your narratives, code, debugging, and screenshots must be your own. Collaboration and the use of AI tools follow the course policy described above.